

Stores and Digital Products

A seller opens a store, publishes a paid digital product, and an anonymous visitor reaches it at a stable public URL and sees its XOF price. The first product vertical — and the first time the money model carries a real price a person typed.

Branch **claude/create-app-repository-1cxsih**

Head **6398b06**

1 commit

CI run **#11 • 3/3 green**

Tests **196 passed • 5 browser**

1

What was built, and what was deliberately not

The milestone is one vertical slice: **seller → store → paid digital product → publish → public URL → visitor sees it with its price**. Everything downstream of that is a later milestone and none of its code is in this diff.

BUILT	NOT BUILT – BY INSTRUCTION				
Stores with a public identity and a slug	CHECKOUT	PAYMENT	CONSENT GATE	WALLET SETTLEMENT	WITHDRAWAL
Paid digital products priced in the existing money model	PAYOUT	IPAYMONEY	REFERRAL EXPANSION	PHYSICAL GOODS	INVENTORY
Publishing, for both	DELIVERY				
A stable public URL per product					
An anonymous public store page and product page					

The product page shows the price and a *disabled* purchase button that says payment is coming. A button that appeared to work would be a screen lying about what the system can do.

SUGGESTED REVIEW TIME: 45-60 MINUTES

Read `packages/core/src/catalog/index.ts` (524 lines — every rule is there) and `catalog.test.ts` (351 lines, 24 tests). Then section 6 of this note, which is the one place I need a decision from you, and section 7, where one of four mutations was not caught.

The journey, as the browser test executes it

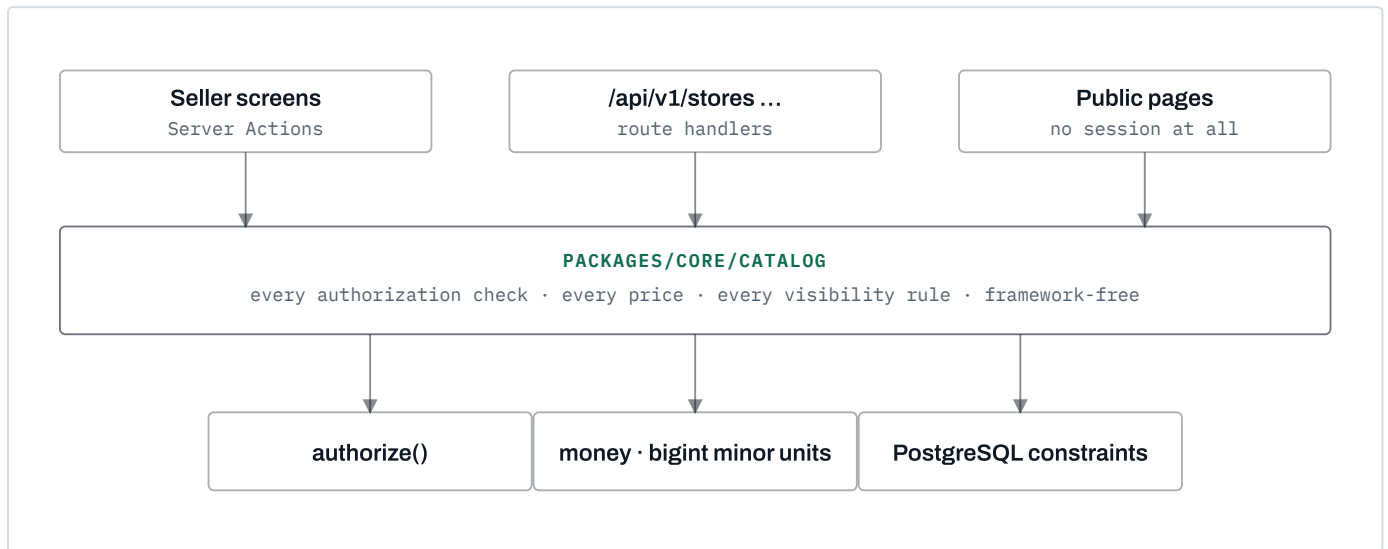
Not a description — this is what runs in CI, in a real browser, against a production build.

#	STEP	WHAT IS ASSERTED
1	Seller signs in by phone	Reaches the wallet; the session resolves to an Afrinext user id.
2	Visits <code>/fr/sell</code> before being a seller	No form is offered — and the API refuses a direct post with 403 , with zero rows written.
3	The <code>seller</code> role is granted	Exactly what an admin console would do; done in SQL because no console exists yet.
4	Creates a store	Redirected to the store's admin page.
5	Publishes the store	A public link appears.
6	Creates a product priced 15 000	The database holds 15000. A 100× error would read 1500000.
7	Anonymous visitor tries the URL <i>before</i> publishing	404 . A draft is not reachable.
8	Seller publishes the product	A second public link appears.
9	Anonymous visitor loads the URL	200 , title and summary visible, price reads exactly 15 000 XOF.
10	The same page is inspected for leaks	The seller's user id and phone number appear nowhere in the HTML.

Step 7 runs in a **separate browser context with no cookies**. "The public can see this" is proved by an actual public request, not by a signed-in page that happens to look public.

One service, two transports

Your instruction was that Server Actions and `/api/v1/*` must ultimately call the same core services, and that no route may hold private business logic. That is the shape:



The transports resolve an actor and render a result. They do not decide anything. If a form post and an API call could disagree about who may open a store or what a valid price is, **one of them would eventually be wrong** — so neither is allowed its own answer.

A price is `bigint` minor units plus a currency code. The exponent comes from the `currencies` table, so XOF's zero decimals are a row rather than a special case: when a seller types 15 000, that is 15 000 francs, and the same string in a two-decimal currency would be 15 000 cents.

GUARANTEE	WHERE IT IS ENFORCED
No float anywhere in the path	<code>parseMoney</code> → <code>Money</code> → <code>bigint</code> column, read back with <code>BigInt()</code> . There is no point at which the amount is a JavaScript number.
A paid product cannot be free or negative	The service refuses it <i>and</i> the <code>products_price_positive</code> check constraint refuses it. A future caller bypassing the service still cannot write one.
The currency must exist and be active	Foreign key to <code>currencies</code> , plus an explicit check — an inactive currency is not a pricing option.
Amounts survive JSON	Minor units cross the API as a <i>string</i> , with the currency and its exponent alongside, so a client never guesses where the decimal point belongs.

THE TEST I WOULD POINT A REVIEWER AT FIRST

One product is priced at 9 007 199 254 740 993 minor units — `Number.MAX_SAFE_INTEGER + 2` — and read back exactly. Round-tripping that through a double lands on ...992. It is a deliberately absurd price whose only job is to fail loudly if anything in this path ever becomes number arithmetic.

A related accident worth keeping: `JSON.stringify` throws on a `bigint`. That forced the transport layer to state how an amount is represented instead of letting one quietly become a float.

Creating a store grants its creator a **store-scoped** `store_owner` role, in the same transaction as the **insert**. Two alternatives were rejected: granting it afterwards leaves a window in which a store exists that nobody can administer, and inferring it at request time from the owner column ("the row says you, so you may") would put an authorization decision outside `authorize()`.

The consequence is tested directly: being a `store_owner` somewhere is not being one everywhere. A seller who owns store A is refused when updating store B or creating a product in it, while still able to work on their own — which is the entire reason roles carry a scope.

The seller screens also check the permission to decide what to render, but that is cosmetic. The Server Action and the API route both check again before doing anything, because **a hidden form is not a permission** — a page can be fetched and an endpoint can be posted to directly. Section 7 is what happens when only the cosmetic check is tested.

WHO MAY OPEN A STORE?

Phase 0 seeded `store.create` as a permission and gave it to **no role at all**, so until now nobody could hold it. That gap had to be closed to build this milestone, and closing it is a policy decision rather than an engineering one.

Granting it to `member` would have decided, on your behalf, that signing up makes you a seller. Instead there is now a `seller` global role carrying *only* `store.create`, and **nothing grants it automatically**. The practical consequence: today there is no self-serve path to becoming a seller — only a direct role assignment.

Three ways forward:

- **Open marketplace** — grant `store.create` to `member`. Anyone who signs up can open a store. Simplest, and the most common model.
- **Manual for now** — keep it as it is until there is an admin console to grant it. Safest, but blocks any real seller onboarding.
- **Gated** — grant it on some verification step. Most work, and needs you to define the step.

I have not chosen. The tests grant the role explicitly, so the vertical is proven either way.

Mutation evidence — one was not caught

DELIBERATE DEFECT	CAUGHT BY	FIRST ATTEMPT
The public query stops requiring the product to be published	Browser test	CAUGHT
Product creation ignores the store scope	13 domain tests	CAUGHT
A price is parsed with the wrong exponent (a 100× error)	Browser test	CAUGHT
<code>createStore</code> stops calling <code>authorize()</code>	1 domain test only	NOT CAUGHT BY THE BROWSER TEST

WHY THE FOURTH ONE SLIPPED, AND WHAT CHANGED

My browser test asserted that a non-seller *sees* no create-store form. With the authorization check removed from the service, the page still consulted the permission to decide what to render — so the form stayed hidden, the test stayed green, and the server would have happily created a store for anyone who posted to the endpoint.

The test now posts directly to `/api/v1/stores` as a signed-in non-seller and asserts `403`, the `authz.denied` code, and **zero rows written**. Re-running the same mutation now fails it.

This is the second time in this project that a test passed on the strength of a mechanism it was not testing. It is the reason mutation runs are worth the ten minutes.

A bug the rebuild-from-zero step caught

Migration 0007 originally created the `seller` role and its permission row. That worked on my development database and **fails on a fresh one**: migrations run before the seed, so the `permissions` row the foreign key points at does not exist yet.

```
error: insert or update on table "role_permissions" violates foreign key
constraint "role_permissions_permission_key_permissions_key_fk"
```

The fix is not a workaround: roles and permissions are reference data, and the seed is where reference data is defined. The migration now creates tables and constraints only, and says in a comment why the role is not there.

This is exactly what the "rebuild the schema from scratch" CI step exists for — a schema that only ever gets migrated forward from a database that already worked hides this class of bug until the first clean deployment.

You required that the public page expose no private seller or administrative data. Two structural choices rather than a filter:

CHOICE	EFFECT
<code>PublicProduct</code> is a separate type , not a filtered copy of the internal record	It has no owner id, no store id, no product id, no status and no timestamps. There is nothing private to leak, because the fields were never selected.
Both visibility conditions are in the WHERE clause	A draft product, or one under an unpublished or suspended store, is not fetched-then-hidden — it is never selected. Suspending a store takes its whole catalogue down without touching a product row.

An unpublished, suspended and non-existent store all answer identically: not found. Telling them apart would let a stranger enumerate which slugs are taken and which sellers are suspended. Store slugs also reserve the paths Afrinext owns — a store called `admin` or `wallet` is refused.

CI run #11 on 6398b06

JOB	RESULT	DURATION
lint · typecheck · test	SUCCESS	1m 48s
browser · authorization assertions	SUCCESS	1m 30s
dependency scan	SUCCESS	20s

```
pnpm test      196 passed | 3 skipped (17 files)  was 172 / 16
playwright test 5 passed                      was 3
lint · typecheck exit 0
build           exit 0
concurrency     6 passed
reconcile-check clean, zero drift
audit          No known vulnerabilities found
```

Files changed — 27, all related

AREA	LINES	WHAT
core/catalog/index.ts	524	The service. Every rule.
core/catalog/catalog.test.ts	351	24 tests.
web/e2e/catalog.spec.ts	199	The journey, plus the anonymous-visitor case.
core/catalog/slug.ts	56	Slug shape and reserved paths.
db/schema/catalog.ts + 0007	144	Two tables, seven check constraints.
web pages and API routes	~570	Four pages, three API routes, forms, actions.
i18n	52	26 keys per locale, appended.
core/authz/permissions.ts	+8	The <code>seller</code> role.
web/lib/api.ts	+3	<code>*_not_found</code> maps to 404 rather than 400.

`pnpm-lock.yaml` is unchanged — no new dependency. Nothing from Phase 1's OTP model, the ledger, or the browser-CI foundation appears in the diff.

ITEM	STATE
Who may open a store (section 6)	NEEDS YOUR DECISION
No consent gate on publishing — scoped to a later milestone	BY INSTRUCTION
No admin console, so the <code>seller</code> role is granted in SQL	ACCEPTED FOR NOW
Migrations remain forward-only; <code>0007</code> has no down-path. CI proves it replays from zero, which is not the same as a rollback.	UNCHANGED, STATED PLAINLY
Digital products only; <code>kind</code> is constrained to <code>'digital'</code> in the database	DELIBERATE
No product file or delivery mechanism — a buyer would have nothing to receive yet	NEEDED BEFORE CHECKOUT IS MEANINGFUL

That last one is worth flagging before milestone 3 is scoped: a "digital product" currently has a title, a summary and a price, but no file, link or access grant attached. Checkout can be built and settled through the ledger without it, but the buyer would receive nothing. Whether the deliverable lands in milestone 3 or after it is a scoping call for you.

Proposed next: milestone 3 — consent presented, versioned, recorded and *blocking*, per your section 5. Then checkout against the mock provider. **Not started.**

Phase 2 · milestone 2 of 5 · branch `claude/create-app-repository-1cxsih` · head `6398b06`